Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
З ЛАБОРАТОРНОЇ РОБОТИ № 4
з дисципліни «Професійний Python»
на тему: «Професійне ООП та система типів Python»

Виконав:
студент 3 курсу
групи ФеП-33
Франков Дмитро
Олександрович
Перевірив:
професор Свелеба Сергій
Андрійович

Львів-2026

1. Мета роботи

Мета роботи: сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

2. Опис предметної області та постановка задачі (Варіант 15)

Предметною областю є автоматизований облік, валідація, статистичний аналіз та експорт результатів наукових експериментів.

Завдання включає:

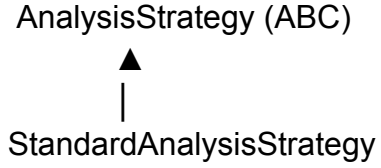
- моделювання сутностей експерименту, дослідника, фізичного вимірювання та результату;
- валідацію вхідних даних та діапазонів допустимих похибок;
- агрегацію експериментів у серію з підтримкою стандартних протоколів ітерації та вимірювання розміру;
- реалізацію стратегії статистичного аналізу (середнє, мінімум, максимум);
- застосування принципів SOLID, Dependency Injection та структурної типізації через Protocol;
- статичну верифікацію кодової бази за допомогою mypy --strict.

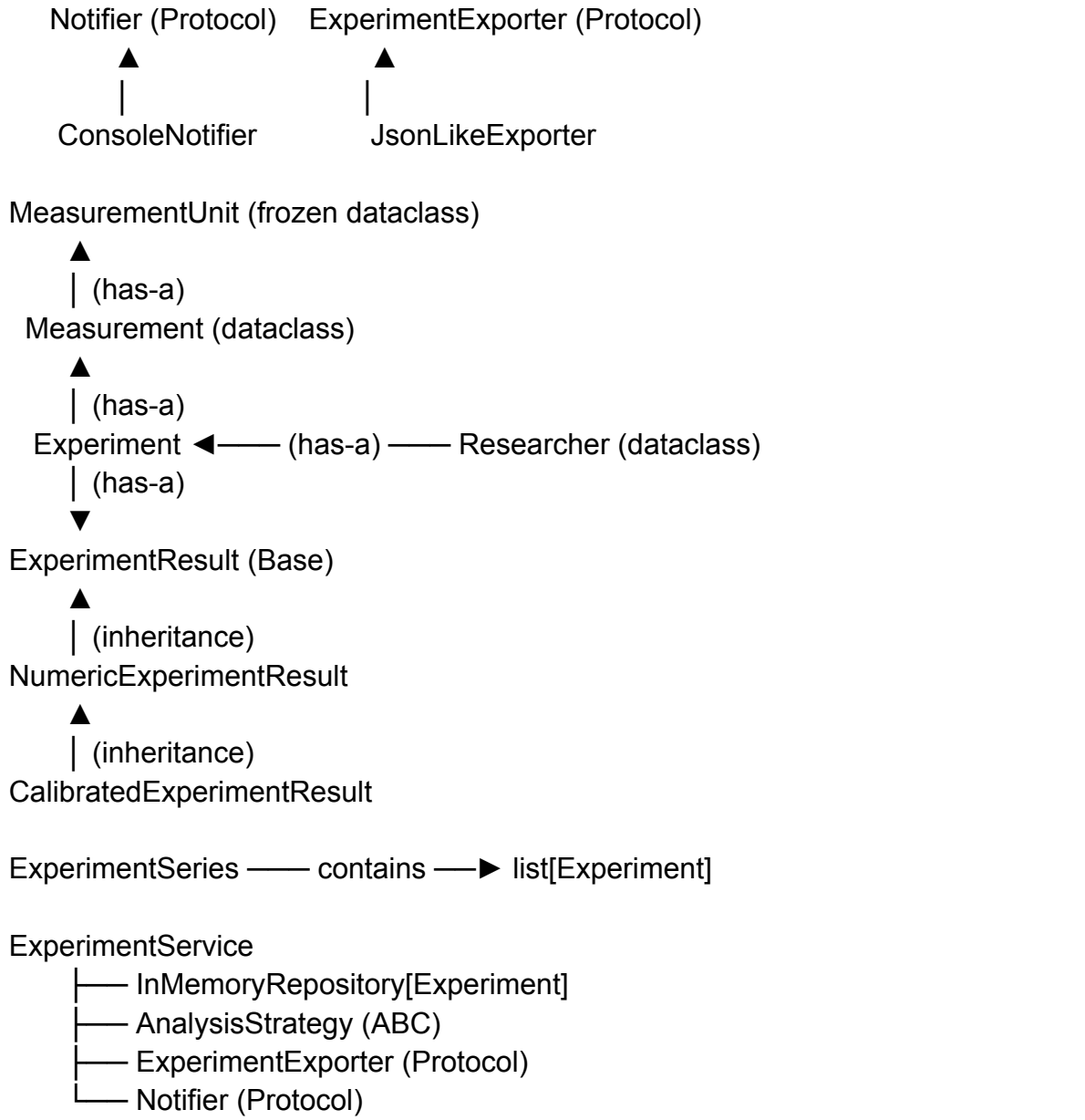
3. Список Domain Entities та Value Objects

1. **MeasurementUnit (Value Object):** незмінний об'єкт значення фізичної одиниці вимірювання.
2. **Researcher (Domain Entity):** дослідник із валідацією імені та контактного email.
3. **Measurement (Domain Entity):** фізичне вимірювання з параметром, числовим значенням та одиницею вимірювання.
4. **ExperimentResult / NumericExperimentResult / CalibratedExperimentResult (Domain Hierarchy):** ієрархія представлення результатів із калібруванням та перевіркою границь.
5. **Experiment (Core Domain Entity):** експеримент, що об'єднує метадані, вимірювання та результат через композицію.
6. **ExperimentSeries (Aggregate Container):** контейнер над списком експериментів із підтримкою протоколу колекцій.

4. Class Diagram

Plaintext





5. Пояснення ключових концепцій ООП та системи типів

- **Inheritance (Наслідування):**
  Клас NumericExperimentResult успадковує ExperimentResult, розширюючи його полями min_bound та max_bound і перевизначаючи логіку методу is_valid(). CalibratedExperimentResult додатково розширює клас, додаючи зсув калібрування через super().__init__(...).
- **Composition (Композиція):**
  Клас Experiment не наслідує дослідника чи вимірювання, а містить їх як частини свого стану (has-a зв'язок).
- **Encapsulation (Інкапсуляція):**
  Прямий доступ до атрибутів захищено через приватні змінні (_title, _raw_value) та методи-аксесори.

- **Property (@property):**
  Забезпечує контрольований інтерфейс до полів із валідацією (наприклад, заборона встановлення порожньої назви експерименту через @title.setter).
- **Dataclass та Frozen Value Object:**
  MeasurementUnit визначено як @dataclass(frozen=True, slots=True), що гарантує його незмінність (immutability) та оптимізує пам'ять.
- **ABC vs Protocol:**
  AnalysisStrategy — це ABC (номінальна типізація, вимагає явного спадкування). ExperimentExporter та Notifier — це Protocol (структурна типізація, duck typing), що дозволяє підключати будь-який сумісний клас без жорсткого зв'язування.
- **Dunder Methods:**
  В ExperimentSeries реалізовано:
  - __len__() — повертає кількість експериментів;
  - __iter__() — дозволяє ітерувати серію у циклі for;
  - __contains__() — оператор in для швидкої перевірки наявності ID.
- **Generic Repository та TypeVar:**
  InMemoryRepository(Generic[T]) параметризовано обмеженим T = TypeVar("T", bound=HasId), що забезпечує безпеку типів при додаванні й отриманні об'єктів з репозиторію.
- **TypedDict:**
  ExperimentPayload(TypedDict) описує форму сирих даних (наприклад, із JSON або API), відокремлюючи зовнішні структури від доменної моделі.

6. Принципи SOLID та Dependency Injection

1. **SRP (Single Responsibility Principle):**
   Experiment зберігає стан експерименту, StandardAnalysisStrategy відповідає виключно за математичні розрахунки, JsonLikeExporter — за серіалізацію, а ExperimentService — за оркестрацію сценарію.
2. **OCP (Open/Closed Principle):**
   Нові методи експорту (наприклад, CsvExporter) або нові стратегії аналізу додаються як нові класи без модифікації коду ExperimentService.
3. **LSP (Liskov Substitution Principle):**
   Будь-який підклас NumericExperimentResult чи CalibratedExperimentResult може бути безпечно використаний замість базового ExperimentResult без порушення контракту.
4. **ISP (Interface Segregation Principle):**
   Протоколи ExperimentExporter та Notifier мають лише необхідні для сервісу сигнатури (export та notify), не змушуючи адаптери реалізовувати зайві методи.
5. **DIP (Dependency Inversion Principle) & Dependency Injection:**
   Клас ExperimentService залежить виключно від абстракцій (InMemoryRepository[Experiment], AnalysisStrategy, ExperimentExporter,

Notifier), а конкретні залежності передаються через конструктор під час ініціалізації.

7. Результати виконання програми (Screenshot Execution)

Plaintext
=== SCIENTIFIC EXPERIMENT SYSTEM (LAB 4) ===
[NOTIFICATION to d.frankov@lnu.edu.ua]: Experiment #3 exceeds boundary tolerance!

--- ExperimentSeries Container Check ---
Total experiments in series (__len__): 3
Is experiment ID 2 present? (__contains__): True
Is experiment ID 99 present? (__contains__): False

--- Iteration over series (__iter__) ---
-> Exp #1 'Quantum Resistance Test' by Dr. Dmytro Frankov -> 12.4500
-> Exp #2 'Thermal Drift Measurement' by Dr. Dmytro Frankov -> 25.1500
-> Exp #3 'Superconductor Overload' by Dr. Dmytro Frankov -> 1500.0000

--- Analysis Strategy Results ---
Count: 3.0000
Average: 512.5333
Min: 12.4500
Max: 1500.0000

Export payload: {"series": "Series Alpha 2026", "experiments_count": 3, "valid_samples": 2}
Repository count: 3

8. Результат перевірки mypy

Команда:

Bash
mypy src

Результат:

Plaintext
Success: no issues found in 8 source files

9. Обов'язковий експеримент з помилками типізації (п. 71 методички)

Навмисно внесено три некоректні типи:

1. Помилка 1: experiment_id: int = "101".

2. Помилка 2: def get_tolerance() -> float: return "0.05".
3. Помилка 3: repo: InMemoryRepository[Experiment] = InMemoryRepository(); repo.add(Researcher(...)).

Діагностика статичного аналізатора mypy:

Plaintext
src/experiment_system/main.py: error: Incompatible types in assignment (expression has type "str", variable has type "int")
src/experiment_system/main.py: error: Incompatible return value type (got "str", expected "float")
src/experiment_system/main.py: error: Argument 1 to "add" of "InMemoryRepository" has incompatible type "Researcher"; expected "Experiment"
Found 3 errors in 1 file (checked 8 source files)

Після фіксації помилок код виправлено на відповідні типи, статус mypy відновлено до Success.

10. Експериментальна частина: порівняння архітектурних рішень (п. 72, 73 методички)

Варіант A vs Варіант B (Dependency Injection)

- **Варіант A (жорстке зв'язування):** сервіс самостійно інстанціює self._exporter = JsonLikeExporter() всередині __init__. Це унеможливлює ізольоване тестування без сторонніх ефектів, порушує DIP та OCP.
- **Варіант B (Dependency Injection):** експортер передається у конструктор як аргумент за контрактом ExperimentExporter. Це знижує зв'язність (coupling), полегшує тестування за допомогою mock-об'єктів і дозволяє легко змінювати формат виведення без редагування сервісу.

Дослідження Inheritance vs Composition

- **Inheritance:** використано для ієрархії результатів вимірювань (ExperimentResult → NumericExperimentResult → CalibratedExperimentResult), де існує чітке співвідношення is-a.
- **Composition:** використано для зв'язку експерименту з дослідником і вимірюванням (Experiment has-a Researcher), оскільки дослідник існує незалежно від конкретного експерименту. Це забезпечує значно менший coupling та легшу повторну використовуваність компонентів.

11. Висновок

У лабораторній роботі №4 спроєктовано та реалізовано професійну типізовану об'єктно-орієнтовану модель предметної області для обліку наукових експериментів. Використання композиції та наслідування дозволило чітко розмежувати відповідальності сутностей. Застосування Protocol та ABC забезпечило гнучку взаємодію компонентів за принципами SOLID, зокрема

через впровадження залежностей (Dependency Injection). Статична типізація за допомогою TypeVar, Generic та TypedDict разом із перевіркою mypy усунула потенційні помилки інтеграції ще на етапі розробки.

Ось повні, чіткі та структуровані відповіді на всі 60 контрольних запитань (розділ 82 методички) для внесення у звіт та підготовки до захисту роботи:

Відповіді на контрольні запитання

1. **Що таке class?**
   Клас — це шаблон або опис структури даних і поведінки (методів), на основі якого створюються конкретні об'єкти.
2. **Що таке object?**
   Об'єкт — це сутність у пам'яті комп'ютера, яка має стан, поведінку та ідентичність, створена за описом класу.
3. **Що таке instance?**
   Екземпляр (instance) — це конкретний створений об'єкт певного класу.
4. **Для чого __init__()?**
   Це спеціальний метод-ініціалізатор (конструктор), який автоматично викликається після створення об'єкта для початкового заповнення його атрибутів.
5. **Що таке instance attribute?**
   Атрибут екземпляра — це змінна, яка належить конкретному об'єкту (зазвичай ініціалізується через self.attr).
6. **Що таке class attribute?**
   Атрибут класу — це змінна, оголошена безпосередньо в тілі класу, яка є спільною для всіх його екземплярів.
7. **Що таке encapsulation?**
   Інкапсуляція — принцип ООП, що поєднує дані та методи їх обробки всередині об'єкта, обмежуючи прямий доступ до внутрішнього стану для збереження його цілісності.
8. **Для чого properties?**
   Декоратор @property дозволяє надавати доступ до методів як до звичайних атрибутів, додаючи логіку читання, валідації чи обчислення.
9. **Чим property відрізняється від public field?**
   Public field дозволяє пряме довільне читання/зміну значення без контролю, а property викликає методи-гетери/сетери з можливістю валідації та заборони некоректних змін.
10. **Що таке inheritance?**
    Наслідування — механізм створення нового класу (нащадка) на основі наявного (базового), що дозволяє повторно використовувати або розширювати логіку.
11. **Що означає relation is-a?**
    Відношення типу «є представником»: об'єкт підкласу є спеціалізованою версією базового класу (наприклад, Manager is an Employee).

12. **Що таке composition?**
    Композиція — підхід у проєктуванні, за якого один клас містить у собі екземпляри інших класів для делегування їм частини роботи.
13. **Що означає relation has-a?**
    Відношення «містить у собі»: об'єкт володіє іншим об'єктом як своєю складовою (наприклад, Car has an Engine).
14. **Коли composition краща за inheritance?**
    Коли класи не мають суворої ієрархічної сутності «is-a», або коли потрібна більша гнучкість, низька зв'язність (loose coupling) і можливість змінювати компоненти в runtime.
15. **Для чого super()?**
    Для коректного виклику реалізацій методів базового класу (зокрема __init__) з урахуванням порядку розв'язання методів (MRO).
16. **Що таке polymorphism?**
    Поліморфізм — здатність різних класів надавати спільний інтерфейс і однаково реагувати на виклики тих самих методів.
17. **Що таке duck typing?**
    Концепція динамічної типізації: якщо об'єкт має необхідні методи й властивості, він вважається придатним для використання незалежно від того, від кого він успадкований («якщо воно крякає як качка...»).
18. **Що таке ABC?**
    Abstract Base Class (модуль abc) — механізм створення абстрактних базових класів, які декларують обов'язковий інтерфейс для нащадків.
19. **Для чого @abstractmethod?**
    Декоратор позначає метод як обов'язковий до перевизначення в дочірніх класах.
20. **Чи можна створити instance abstract class?**
    Ні, Python підніме помилку TypeError, якщо клас має хоча б один неперевизначений @abstractmethod.
21. **Що таке Protocol?**
    Інструмент модуля typing для опису структурних інтерфейсів (structural subtyping).
22. **Чим Protocol відрізняється від ABC?**
    ABC вимагає явного спадкування (номінальна типізація), а Protocol перевіряє лише наявність відповідних сигнатур методів без явного наслідування (структурна типізація).
23. **Що таке structural typing?**
    Сумісність типів, що визначається структурою об'єкта (набором методів і полів), а не його оголошеним місцем в ієрархії спадкування.
24. **Що таке nominal typing?**
    Сумісність типів, що визначається виключно явним оголошенням та ієрархією класів.

25. **Що таке dataclass?**
    Декоратор @dataclass, який автоматично генерує стандартні службові методи на основі анотацій типів атрибутів.
26. **Які methods генерує dataclass?**
    За замовчуванням генерує __init__(), __repr__(), __eq__(), а також за потреби __hash__(), __lt__() тощо.
27. **Для чого default_factory?**
    Для безпечної ініціалізації змінюваних (mutable) значень за замовчуванням (списків, словників) новим екземпляром для кожного об'єкта.
28. **Чому mutable default є проблемою?**
    Один і той самий екземпляр списку чи словника буде спільним між усіма об'єктами класу, що призведе до непередбачуваних мутацій даних.
29. **Для чого __post_init__()?**
    Метод у dataclass, який виконується одразу після згенерованого __init__() для валідації стану або обчислення залежних полів.
30. **Що означає frozen=True?**
    Робить екземпляр dataclass незмінним (immutable); спроба модифікації полів викликає помилку FrozenInstanceError.
31. **Що означає slots=True?**
    Використовує механізм __slots__, що запобігає створенню внутрішнього словника __dict__ для кожного екземпляра, заощаджує пам'ять та пришвидшує доступ до атрибутів.
32. **Що таке dunder method?**
    «Double underscore» або магічний метод (наприклад, __str__), через який клас інтегрується зі стандартними протоколами Python.
33. **Для чого __repr__()?**
    Повертає точне й однозначне рядкове представлення об'єкта, орієнтоване на розробників і налагодження.
34. **Для чого __str__()?**
    Повертає читабельне, зручне для користувача рядкове представлення об'єкта під час виклику print() або str().
35. **Для чого __len__()?**
    Дозволяє застосовувати до об'єкта вбудовану функцію len().
36. **Для чого __iter__()?**
    Повертає ітератор над колекцією, дозволяючи обходити об'єкт у циклі for.
37. **Для чого __contains__()?**
    Реалізує перевірку входження за допомогою оператора in (наприклад, x in container).
38. **Для чого __call__()?**
    Дозволяє викликати екземпляр класу як звичайну функцію: my_object().
39. **Що таке SOLID?**
    Абревіатура п'яти фундаментальних принципів об'єктно-орієнтованого проєктування для створення підтримуваних і розширюваних систем.

40. **Що означає SRP?**
    Single Responsibility Principle — кожен клас повинен мати лише одну причину для зміни (одну зону відповідальності).
41. **Що означає OCP?**
    Open/Closed Principle — програмні сутності мають бути відкритими для розширення, але закритими для модифікації.
42. **Що означає LSP?**
    Liskov Substitution Principle — об'єкти підкласів повинні без порушення контракту замінювати об'єкти базового класу.
43. **Що означає ISP?**
    Interface Segregation Principle — клієнти не повинні залежати від інтерфейсів чи методів, якими вони не користуються (краще декілька спеціалізованих інтерфейсів).
44. **Що означає DIP?**
    Dependency Inversion Principle — модулі високих рівнів не повинні залежати від модулів низьких рівнів; обидва мають залежати від абстракцій.
45. **Що таке dependency injection?**
    Впровадження залежностей — патерн, за якого зовнішні компоненти/сервіси передаються в об'єкт ззовні (через конструктор), а не створюються ним самостійно.
46. **Чому DI покращує testability?**
    Дозволяє легко підміняти реальні залежності (БД, мережу) на mock-об'єкти чи заглушки під час ізольованого тестування.
47. **Що таке type hint?**
    Анотація типів у сигнатурах функцій і змінних, яка документує очікувані типи для IDE та статичних аналізаторів.
48. **Чи перевіряє Python type hints автоматично в runtime?**
    Ні, інтерпретатор Python ігнорує type hints під час виконання, зберігаючи динамічну типізацію.
49. **Що таке static type checker?**
    Утиліта (наприклад, mypy або pyright), яка аналізує код до запуску без виконання самої програми та виявляє помилки невідповідності типів.
50. **Для чого mypy?**
    Офіційний інструмент статичної перевірки типів для мови Python.
51. **Для чого Pyright?**
    Швидкий статичний аналізатор типів від Microsoft (часто інтегрований у VS Code).
52. **Що таке TypeVar?**
    Конструктор змінної типу, що використовується для написання узагальненого (generic) коду.
53. **Що таке Generic?**
    Клас або функція, які можуть працювати з різними типами даних, зберігаючи строгий зв'язок між типами параметрів.

54. **Що означає Generic[T]?**
    Вказує, що клас є узагальненим і параметризований змінною типу T.
55. **Що таке bound TypeVar?**
    Обмеження змінної типу: TypeVar("T", bound=ParentClass) дозволяє передавати лише ParentClass або будь-який його підклас.
56. **Що таке constrained TypeVar?**
    Фіксований перелік дозволених типів: TypeVar("T", int, float) дозволяє лише явно зазначені типи.
57. **Що таке TypedDict?**
    Спеціальний тип словника для статичного аналізу, що описує очікувані ключі та типи їхніх значень у dict.
58. **Чим TypedDict відрізняється від dataclass?**
    TypedDict під час виконання залишається звичайним словником Python (dict), тоді як dataclass є повноцінним екземпляром користувацького класу зі своїми методами та валідацією.
59. **Коли доцільно використовувати TypedDict?**
    Для типізації неструктурованих даних, JSON-відповідей API, конфігурацій та сирих мапінгів, які надходять ззовні.
60. **Чому external payload і domain object краще розділяти?**
    Для ізоляції доменної бізнес-логіки від зовнішніх форматів і змін в API; сирі дані мають перевірятися й конвертуватися в безпечні сутності предметної області на вході в систему.